TECHNICAL APPROACH DOCUMENT

CastIron Media — Phase 1 Lakehouse

This document sets out the recommended architecture for CastIron Media's Phase 1 lakehouse on Databricks and AWS, covering streaming ingest, identity resolution, measurement, the Omnicom clean room, and reverse-ETL, delivered through modular, reusable Terraform.

1. Purpose and Scope

This document presents the recommended Phase 1 architecture for CastIron Media's lakehouse platform on Databricks and AWS — the single approach that best fits the brief's requirements, not a comparison of alternatives. It covers streaming ingest, identity resolution, measurement, the Omnicom clean room, and reverse-ETL: how each is delivered, how it performs, what it costs to run, and what still needs confirming before build. This is the architecture and its Terraform foundation, not executable pipeline code — Section 9 lists what is needed from CastIron before the pipeline business logic itself can be built.

> **Key notes**
>
> **Billing:** Databricks is subscribed through AWS Marketplace and billed under the AWS account invoice. There is no separate Databricks bill.
>
> **Infrastructure:** provisioned with modular, reusable Terraform across dev, staging and production (Section 7).
>
> **Clean room:** CastIron registers its own Iceberg data through its own Snowflake provider account in the same AWS region as the partner, then links a read-only offering into the partner's Collaboration Data Clean Room via the Unity Catalog Iceberg REST Catalog — zero-copy. See Section 6.
>
> **Costs:** approximate list prices (AWS us-east-1, Databricks Premium tier) for planning only.

2. Current State

The entire platform runs on AWS today. Ad-server events flow through Kinesis and Firehose into Redshift Serverless, where a single stored procedure performs all transformation before the reporting application queries Redshift over JDBC.

Component	Current implementation	Limitation
Ingest	Kinesis Data Streams (~6,500 events/s; 562.7M/day, 736M peak) to Firehose COPY into Redshift	Event column contract duplicated in five places, one only in the Firehose console
Transformation	~950-line stored procedure: normalization, geo and device enrichment, request-to-event join, dedup, late-event retry, attribution, reach	Unversioned, untested, monolithic; no lineage or data-quality checks
Consolidation	Trailing 1 to 2 day re-aggregation for 48 h partner restatements	Full-window recompute
Serving	SPA via JDBC: ~15 packaged reports and a self-serve report builder	Tied to Redshift

Orchestration	Custom SQS + Valkey scheduler (67 tasks, ~30 pipelines)	Bespoke, no observability
Storage	Redshift ~500 GB active; S3 archive ~100 TB; Aurora MySQL for application data	No governance catalog

3. Phase 1 Requirements

Workstream	Type	Requirement	Volume / constraint
Streaming ingest	Migration	Rebuild the event pipeline on Databricks, run in shadow alongside Redshift until output parity is proven, then cut over and retire the Redshift path	24/7; 562.7M events/day; 48 h late data
Identity resolution	Greenfield	Daily match of device events to households using a licensed third-party device graph (hosted by licensor or delivered resident)	8M probes/day vs 3.2B identifiers; resident graph ~0.79 TB
Measurement	Greenfield	Exposure records, 30-day rolling household reach and frequency, equivalization, partner delivery reconciliation, third-party validation ingest	~250M exposure records/month; 48 h restatement
Clean room	Greenfield (hard requirement)	Share contribution data with the agency partner's Snowflake clean-room tenant — zero-copy Iceberg interoperability	No row-level data leaves CastIron's environment
Reverse-ETL	Greenfield	Write curated results back to Aurora MySQL	No impact on application performance
Reporting	Migration	Serve the SPA's ~15 reports and self-serve builder from the new platform after cutover	Interactive response times

4. Recommended Architecture

Streaming runs continuously on classic compute for the lowest steady-state unit cost; the licensed identity graph is held resident in Unity Catalog for the fastest daily join; measurement and clean-room contribution run as scheduled batch jobs against the governed lakehouse; and the SPA is served from a serverless SQL warehouse that scales and stops on demand. Terraform deploys every layer — AWS foundation, the Databricks workspace, governance, and the Snowflake interoperability layer — as separately staged, version-tagged modules.

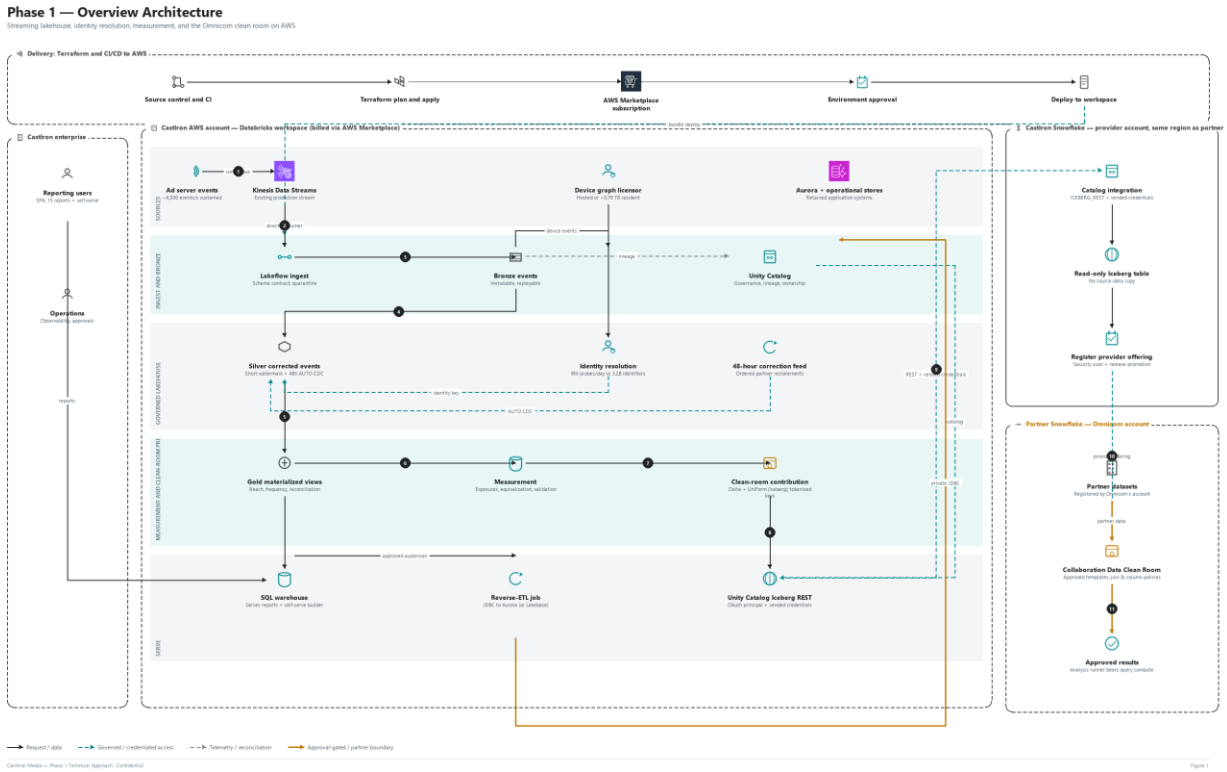


Figure 1. Phase 1 overview architecture — sources, governed lakehouse, clean room, and delivery

4.1 How each requirement is delivered

Phase 1 requirement	How this architecture delivers it
Streaming ingest	A dedicated Kinesis enhanced fan-out (EFO) consumer feeds Lakeflow Declarative Pipelines (bronze, silver, gold) on classic compute with Photon. Stored-procedure logic is split into tested steps: schema contract, geo and user-agent enrichment, watermarked request-to-event join, dedup, 48 h late-data reprocessing. Runs in shadow; a parity job compares outputs with Redshift before cutover, report by report.
Identity resolution	The licensed device graph is delivered resident into Unity Catalog, clustered on device ID. A daily Photon join matches 8M probes against the 3.2B-row identifier table; results are exposed through a stable view so the licensor can be changed without rework.
Measurement	An exposure table plus household-day aggregate produce exact 30-day reach and frequency with versioned equivalization rules. Partition-level reprocessing handles 48 h partner restatements; Auto Loader ingests third-party validation files.
Clean room	Contribution tables are written as Delta with UniForm (Iceberg metadata) and exposed through the Unity Catalog Iceberg REST Catalog. CastIron's own Snowflake provider account — provisioned in the same AWS region as the partner's account — reads them via a catalog integration into a read-only Iceberg table, then registers and links a provider offering into the partner's Collaboration Data Clean Room under approved query templates and join/column policies. No row-level data is ever queried directly by the partner's account.
Reverse-ETL	Hourly Spark JDBC upserts into Aurora MySQL with throttled parallelism; Lakebase synced tables are a viable alternative once the target is Postgres-compatible.
Reporting and orchestration	A serverless SQL warehouse serves the SPA's reports and self-serve builder from governed gold marts; Lakeflow Jobs replaces the custom SQS + Valkey scheduler.

5. How It Performs

Metric	Expected performance
Event to silver latency	1 to 2 minutes
Event to report marts (gold)	5 to 10 minutes
Burst handling	Autoscaling to ~25K events/s (3x peak) — subject to Kinesis shard sizing, see Section 8
Identity daily match (8M × 3.2B)	~30 to 45 minutes
30-day reach and frequency refresh	~1 to 2 hours daily
SPA packaged reports	p95 2 to 5 s (cached < 1 s); warehouse resumes in seconds
Self-serve builder	Sub-second to a few seconds
Reverse-ETL freshness	Hourly
Recovery	Automatic restart from checkpoints; full replay from bronze

6. Clean Room — Design and Cost

The clean room is a hard requirement, not an optional integration, so it is treated as its own section rather than a line in the architecture table. CastIron does not give the agency partner's Snowflake account any direct read access to CastIron's data. Instead, CastIron registers its own Iceberg data through its own Snowflake provider account — required to be in the same AWS region as the partner's account — and links only a read-only, policy-governed offering into the partner's Collaboration Data Clean Room.

- Catalog integration: ICEBERG_REST + vended, short-lived credentials — no personal access tokens, no long-lived secrets.
- Read-only Iceberg table: no source-data copy ever lands in Snowflake-managed storage.
- Registered provider offering: security-scanned and release-promoted before the partner can link it.
- Partner-side Collaboration Data Clean Room: approved query templates and join/column policies bound the partner to aggregate output only.

This is billed separately from the AWS/Databricks run cost above: approximately USD 500 to 1,200 per month (base ≈ USD 1,015/month) for CastIron's own DCR application and analysis warehouse credits, cloud-services allowance, and result/metadata storage — confirm against the Snowflake order form before approval.

7. Delivery Approach

Infrastructure is built from reusable, version-tagged Terraform modules composed into layered stacks per environment. Each layer keeps its own state, so a clean-room change cannot affect the core lakehouse, and a lakehouse change cannot affect the clean-room interoperability layer. Data pipelines and jobs are deployed separately through Databricks Asset Bundles.

Layer	Reusable modules	Notes
L0 Bootstrap	State bucket and locking, CI roles (OIDC), state encryption key	First apply, before any workload exists
L1 AWS foundation	aws/network, aws/kms, aws/storage, aws/iam-databricks, aws/kinesis-consumer	Databricks subscription via AWS Marketplace is a prerequisite
L2 Databricks account	databricks/workspace, databricks/serverless-network	Workspace, private connectivity, identity

L3 Governance	databricks/unity-catalog, databricks/external-sharing (Iceberg REST access)	Unity Catalog, lineage, external access controls
L4 Workload config	databricks/identity, databricks/compute-policies, databricks/observability	Cluster policies, instance pools
L5 Interoperability	Iceberg REST exposure, Snowflake provider objects, clean-room registration	Isolated state — a clean-room failure cannot touch L2/L3

- Pinned Terraform and provider versions; semver-tagged modules; remote S3 state per stack and environment (encrypted, versioned, locked).
- CI runs format, validate, lint, security scan and plan on every pull request; production apply requires approval.
- Mandatory cost tags on all resources; no static credentials (OIDC for AWS, OAuth service principals for Databricks).

8. Cost

Workload	Resource and sizing	Usage basis	Databricks (USD/mo)	AWS (USD/mo)	Total (USD/mo)
Event ingest	Kinesis EFO consumer, 12 shards	24x7; ~12.6 TB read	-	300	300
Streaming pipeline	Lakeflow Declarative Pipelines (Advanced), classic + Photon; driver + 4 x r6gd.2xlarge	24x7; ~10 DBU/h	2,630	1,210	3,840
Late data and 48 h restatement	Jobs compute; driver + 4 x r6gd.xlarge (spot)	~2 h/day	50	40	90
Identity resolution	Jobs compute + Photon; driver + 8 x r6gd.2xlarge (spot); graph on S3 (0.79 TB)	~45 min/day	120	60	180
Measurement	Jobs compute + Photon; driver + 4 x r6gd.2xlarge (spot)	~3 h/day	270	110	380
Clean-room contribution	Jobs compute; driver + 2 x r6gd.xlarge; partner read requests	~1 h/day	15	35	50
Reverse-ETL	Jobs compute from instance pool; driver + 2 x r6gd.xlarge	Hourly, ~5 min/run	30	30	60
SPA reporting	Serverless SQL warehouse, Small (12 DBU/h), 1 to 2 clusters, auto-stop	~12 active h/day	3,020	-	3,020
Lakehouse storage	S3 Standard (Delta bronze, silver, gold)	~23 TB avg + requests	-	630	630
Network and security	NAT (2 AZ), VPC/PrivateLink endpoints, KMS, CloudWatch	24x7	-	300	300
Non-production	Dev + staging (~20% of production compute)	Auto-terminate	700	300	1,000
Total (steady state)			6,835	3,015	~9,850

Annual run cost: approximately USD 118,000 (steady state), plus about USD 1,200 for the shadow period. This excludes the separately billed Snowflake clean-room account priced in Section 6 (~USD 6,000 to 14,400/year).

The supplied brief sizes steady-state spend at approximately USD 213,000 ARR. The figures above are the Databricks-plus-AWS-consumption component of that number, billed under a single AWS Marketplace invoice; confirm what the brief's ARR estimate already includes (Snowflake, services, ramp timing) before treating the gap as fully reconciled.

9. Open Items to Confirm Before Build

- Kinesis capacity mode, shard count, retention, and consumer API. 12 shards alone support ~12K records/s — reaching the ~25K/s burst target needs ~25+ shards, on-demand mode, or producer-side record aggregation.
- The partner's Snowflake account region, and which Collaboration Data Clean Rooms application/version Omnicom runs — the clean-room SQL's procedure calls must be confirmed against that specific installed version before relying on them beyond a dry run.
- Whether CustomerLake Profile Agents can resolve device IDs and households, or only person-entity matches (name/email) — this determines whether Profile Agents can accelerate identity resolution alongside, or instead of, a licensed third-party device graph.
- The final identity licensor selection, and whether its licence terms permit graph-derived identifiers to flow into a partner clean room.
- Certified measurement definitions: the equivalization methodology, the exact household reach/frequency de-duplication and windowing rules, the partner delivery reconciliation tolerance, and the shadow-run parity acceptance threshold. None of these are specified yet beyond the structural requirements in Section 3.
- Reverse-ETL target: Spark JDBC to Aurora MySQL (as priced above) versus Lakebase synced tables — Lakebase is Postgres-compatible, so a MySQL application target still needs the JDBC path, or a migration, to use it. Also needed: the real Aurora schema, expected write volume, the acceptable performance impact/SLO on the application database, and the natural key for idempotent upserts.
- PII and consent handling for device IDs and IP addresses under applicable privacy law — retention limits, opt-out handling, and tokenization before data reaches the clean room.